Republic of Tunisia

Ministry of Higher Education

and Scientific Research

University of Carthage

National Engineering School of Carthage

End of Studies Project Report

By

Youssef HASNAOUI

Trace & Debug Features in STM32 Microcontrollers

Professional Supervisor: **Mohamed HAMROUNI**

Academic Supervisor: **Faten Salem**

Project Elaborated Within STMicroelectronics

Academic Year 2025 - 2026

Dedications

I dedicate this work to my family and friends.

*To my mother **Monia Sellami**, your love and support means so much to me in every step of my journey. You're the best.*

*To my father **Makram Hasnaoui**, thank you so much for all the sacrifices you've made for my sake. I could never be here without you.*

*To my sister **Aicha**, your kindness and your belief in me have been a constant source of inspiration, and for that, I am very grateful.*

To my friends, you have my most sincere gratitude for your support, your guidance and your company throughout this journey.

Thank you,

Youssef HASNAOUI

Acknowledgements

I would like to sincerely thank everyone who had a hand in the development of this project.

Firstly, I want to express my utmost appreciation to ***Mohamed HAMROUNI****, his guidance and support has been invaluable, and has kept me motivated and determined to give my best effort.*

I would also like to thank my professor ***Faten SALEM*** *for providing the essential foundation to pursue this domain of expertise and for her captivating curiosity, which insipred me to continuously expand the boundaries of my knowledge.*

Last but not least, to every single person that contributed to the successful completion of this project with their constant support and motivation, you have my utmost thanks.

Contents

List of Figures

List of Tables

Introduction

Embedded systems operate under real-time constraints while interacting directly with hardware peripherals and asynchronous external events, this makes them inherently difficult to observe and debug. Traditional techniques such as serial logging or software breakpoints are often either insufficiently informative or intrusive enough to perturb the behavior. To address these limitations, modern ARM-based microcontrollers integrate advanced hardware debug and trace capabilities through the ARM CoreSight architecture, enabling non-intrusive visibility into instruction execution, memory accesses, and system activity.

STM32 microcontrollers expose a substantial subset of these capabilities.But, Despite the sophistication of the underlying hardware, these features remain largely locked behind proprietary tooling ecosystems or surface only as limited workflows exposed by commercial IDEs. Within mainstream open-source debugging environments, instruction trace is rarely treated as a first-class debugging primitive and instead exists as a fragmented collection of low-level mechanisms requiring significant manual integration effort.

In practice, utilizing instruction trace frequently involves extracting raw trace data through custom scripts, decoding packets through standalone frameworks such as OpenCSD, and relying on separate visualization or analysis frontends disconnected from the debugging workflow itself. The resulting process lacks standardization, interoperability, and accessibility, effectively placing advanced trace-assisted debugging outside the reach of many embedded developers despite the hardware support already present on modern devices.

This thesis addresses that gap through a study of the trace and debug infrastructure available on STM32 microcontrollers, spanning CoreSight components, transport protocols, debug probes, software tooling, and host-side integration mechanisms. The work then proposes a practical and reproducible workflow for instruction trace capture, decoding, and visualization, designed to integrate with existing debugging environments while remaining independent of proprietary IDE abstractions.

Chapter 1

Context & Problem Statement

Contents

1.1 Host Organization

1.1.1 STMicroelectronics

STMicroelectronics (ST) is a global leader in the semiconductor industry, providing innovative solutions across various markets, including automotive, industrial, personal electronics, and communications equipment.

Founded in 1987 through the merger of SGS Microelettronica of Italy and Thomson Semiconducteurs of France, ST has grown to become one of the largest semiconductor companies in the world.

1.1.2 Activity Sectors

ST's mission is to be the undisputed leader in the semiconductor market, delivering sustainable and innovative solutions that make a positive impact on people's lives. The company's vision is to create technology that enables a more intelligent and connected world, driving progress in key areas such as smart driving, smart industry, smart home, and smart city applications.

The following illustration highlights the key end markets targetted by ST's solutions.

Figure 1.1: STMicroelectronics Activity Sectors

1.1.3 Global Presence

STMicroelectronics operates in more than 35 countries, with a strong presence in key markets around the world. Figure 1.2 illustrates the company's global network, which includes **st_comp**:

- **Manufacturing Sites:** ST has 14 main manufacturing sites, strategically located to serve its global customer base efficiently.
- **Sales & Marketing Offices:** With a network of sales and marketing offices, ST provides localized support and services to its customers.
- **R&D Centers:** ST's R&D centers are spread across the globe, fostering innovation and collaboration.

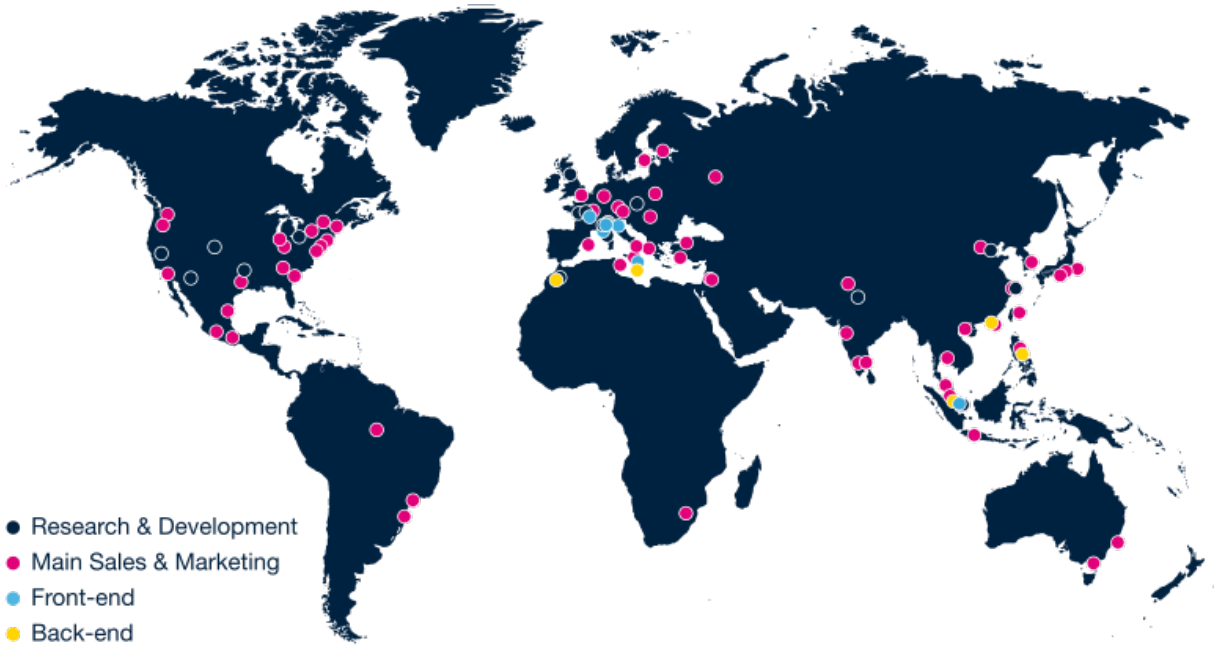

Figure 1.2: STMicroelectronics Worldwide Presence

1.1.4 ST Tunis

The STMicroelectronics Tunis site, located in the El Ghazela Technopark, employs over 100 professionals in various fields such as tools, applications, quality, and support functions. The facility is involved in all stages of the microelectronics industry, from initial circuit design to final verification before mass production.

With expertise in both hardware and software, the Tunis center's activities include electronic circuit design, software tool development, application software support, and the electrical validation and verification of new circuits. This diverse skill set allows the center to contribute significantly to ST's global operations.

Figure 1.3: STMicroelectronics Tunis

1.1.5 Application & Product Support Team

The Application & Product Support team is dedicated to delivering comprehensive customer support while managing and publishing STM32 product documentation. Their role includes providing in-depth technical support for ST's tools and products, helping customers troubleshoot problems and optimize their use of these technologies. They ensure the accuracy, clarity, and consistency of all technical documentation, which is regularly updated to reflect the latest developments and innovations. The team is also involved in enhancing and maintaining existing products, as well as defining new products, derivatives, and application references to meet evolving market needs. Additionally, they lead initiatives related to intellectual property applications, offering specific support through the development of application notes, training programs, and other resources that help customers fully understand and utilize ST's products. This multifaceted approach ensures both customer satisfaction and continuous improvement in the STM32 ecosystem.

1.2 Project Context

1.2.1 Problem Statement

Debugging software is designed to offer a unified experience across a wide range of chips, vendors and architectures, for this, it relies on a common layer of abstraction that groups these heterogeneous targets behind a single conceptual model, This approach is valid for simple primitives such as memory views and breakpoints, however, the shortcomings appear when we move to more advanced features, such as reconstructing execution flow, conditional and cross-triggering of debug events, or recovering execution history after the system failed. These features do not generalize properly, and are tightly coupled to vendor-specific implementations.

Further more, the current debugging workflows stop at the presentation of symptomps, A variable having unexpected values, or the code execution reaching an undesired path, while these views provide observability over the system, they do not explain why a fault has happened in the first place, observability and root-cause analysis remain separate concerns, and this is reinforced by the fact that the notion of an "erroneous state" has no formal standing in the debugging process.

The gap between what the silicon can do and what the developer can actually reach, alongside the lack of formality behind the definition of an erroneous state are the central concerns of this thesis.

1.2.2 State of the Art

Debugging an embedded system from a host machine needs multiple tools to work together in order to achieve the desired goal: Beyond the debug interface present on the microcontroller itself, the user needs a debug probe physically connected to the device, and software on the host machine called a debugger. Generally, the entire

workflow is is tightly coupled from a specific vendor, so it's worth looking at the state of the art based on vendors.

Table 1.1: Embedded Systems IDE Vendors

Vendor	Toolchain	Debugger	Debug Probe
IAR	IAR toolchain	C-Spy	I-Jet/I-Jet Trace
Keil	ARMCC	Keil µVision Debugger	ULink Family
Arm	ARMCC	Arm Debugger	Arm DSTREAM family
Segger	Segger toolchain	Ozone	J-Link/J-Trace
STMicroelectronics	ST-Clang	ST-OpenOCD	STLink
Free	GCC/LLVM Clang	OpenOCD/PyOCD	CMSIS-DAP

It is also worth looking at the features provided by the proprietary debuggers for CoreSight based debug infrastructures:

Note: ST's tools are specialized forks of open-source projects, therefore won't be included in the comparison

Table 1.2: Debugger Features

Feature	IAR	Keil	Arm	Segger
ITM trace	✓	✓	✓	✓[1]
Instruction Trace	✓	✓	✓	✓
CTI support	✗	✗	✓	✗
Scripting Interface	✓	✓	✓	✓
Multi-Core Debugging	Limited	Limited	✓	Limited
IDE coupling	✓[2]	✓	✓	✗

[1] Supports UART encoding only
[2] Implemented in the IDE but can work as a standalone server

1.2.3 Critique of Current State of the Art

While toolchain vendors aim to provide many features within their IDEs, and to a degree, an extensible platform via the scripting interface. However, the main limitation comes in the form of the UI coupling. The fact that the UI, or the debugger's state needs to be aware of the state of the debuggee limits the capabilities of the scripting interface, as with CoreSight components, programming happens via a memory-mapped interface, and a debugger generally has no idea about the component or register behind the address of a read/write command.

Diving more into details, IDEs generally provide a set of configurable settings for tracing capabilities, and no configuration beyond that is possible. More Specifically, when executing a program in a debug environment, the debugger will perform a sequence of actions which includes overriding the configuration registers of CoreSight components with preset values. This makes any extending behaviour on proprietary tools pretty much impossible. Below is a list of unacheivable features:

Table 1.3: Debugger Limitations

Feature	Description
Conditional Tracing	Conditional tracing relies on managing the resources of the ETM to start/stop or include/exclude portions of the execution history.
Event tracing	Events are elements that signal the occurence of specific conditions related to the execution and/or state of the application, they can be used to infer more specific details about the runtime context.
CTI awarness	The CTI is a component that allows the propagation of events across many components, this can for example allow for the synchronization of halting multiple cores, or generating interruputs from within the debug elements.

It is also worth looking at the limitations behind debug probes, as a note, each toolchain vendor, that being IAR, Keil, and Segger provide two versions of their probes, a base model, and an extended one capable of capturing trace via dedicated trace pins.

Note: due to not being able to obtain and work with the DSTREAM family, it will be ommitted from this comparison.

Table 1.4: Debug Probe Limitations

Probe	IAR	Keil	Segger	Trace Capture
I-Jet	✓	✗	✗	✓[1]
ULink	✗	✓	✗	✓[1]
J-Link/J-Trace	✓	✓	✓	✓[1]
STLink	✓	✓	✓	✗
CMSIS-DAP	✓	✓	✓	✗

[1] Given the appropriate model

1.2.4 Proposed Solution

1.3 Conclusion

Abstract

This report describes the work carried out during my summer internship at STMicroelectronics, focused on platform-agnostic benchmarking. The main objective was to run Coremark on STM32 devices using Csolution project structure. To achieve this, I put to use my knowledge in software and embedded development,C code compilation and code generation, using tools such as CMSIS-Toolbox, C and Golang. The results helped to drastically reduce the time required to setup and run Coremark projects using multiple compilers, as well as generating the most optimal results. This internship also allowed me to strengthen my skills in the compilation process, embedded projects architecture design and gain insight into the specific challenges of embedded systems engineering.

Keywords : C/C++, Golang, Embedded Systems, STM32, Compilation, Code Generation.

Résumé

Ce rapport décrit les travaux réalisés lors de mon stage d'été chez STMicroelectronics, axé sur l'analyse comparative indépendante de la plateforme. L'objectif principal était d'exécuter Coremark sur des microcontrôleurs STM32 en utilisant la structure de projet Csolution. Pour ce faire, j'ai mis à profit mes connaissances en développement logiciel et embarqué, en compilation et génération de code C, à l'aide d'outils tels que CMSIS-Toolbox, C et Golang. Les résultats ont permis de réduire considérablement le temps nécessaire à la configuration et à l'exécution des projets Coremark avec plusieurs compilateurs, tout en optimisant les résultats. Ce stage m'a également permis de renforcer mes compétences en compilation, en conception d'architecture de projets embarqués et de mieux comprendre les défis spécifiques de l'ingénierie des systèmes embarqués.

Mots clés : C/C++, Golang, Systèmes embarqués, STM32, Compilation, Génération de Code.